BSc (Hons) in **Computer Science**

Faculty of Computing

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P – Performance Measure: the objective metric that grades environmental success
E – Environment: the external world state, contexts, and systems the agent inhabits
A – Actuators: the physical/software mechanisms the agent triggers to act
S – Sensors: the hardware/API ports through which the agent receives percept sequences

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

Environment component variables are as follows, self.width and self.height are used to set up the boundaries of the grid, self.walls keeps the positions of the walls, self.food_positions keeps the locations of food, and self.opponents keeps the positions of other dynamic objects in the world.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

The present environment should therefore be categorized as Multi Agent as the self.opponents actually refers to other entities who perform their actions freely using execute_action and move, thus possibly colliding with the agent and influencing the score by means of the -50 penalty for collision, which indicates that the result is determined not only by the actions of the agent itself.

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete, cumulative historical log of every percept an agent has received from the moment it was initialized up to the current time step.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

The get_percept() method represents the Sensors component of the PEAS framework, since it is the mechanism through which the agent receives its percept data from the environment.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

In this environment, it is only possible to obtain partial information. The agents can use the return value of the function get_percept() to determine its own position, the positions of its opponents, whether or not it can smell food, whether it has hit a wall, and how many food items are still left to pick up. However, there is no way for the agent to derive the complete state of the environment since certain information about the walls of the environment and the existence of food in other positions, for instance, is never revealed to the agent.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

When execute_action() deducts points for hitting a wall, it is updating the Performance Measure component of PEAS.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

The point mentioned above is significant in a structural sense as it is important that the performance measure should measure the actual alterations of the external environment rather than the agent's internal computation; otherwise, the measure can be exploited. If the measure indicates only internal processing effort instead of any real events occurred in the real world, then a rational agent can do something with the help of this measure without affecting the real world, very much like it is possible to exploit a reward that is based on the motor activity instead of determining how clean the agent really is.

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The agent's awareness of the placement of traps in the environment is influenced by self.toxic_traps being added to the environment but concealed from its viewpoint. Since the agent lacks any perception of the presence of traps in the environment, it cannot integrate any possible trap information into its decision making, rendering the environment ever more partially observable.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

The sensor "smells_toxin" provides the agent with additional perception sequences based on a real clue about the dangers present in its environment associated with a concrete cost that the environment incurs. Rationality is about maximizing the expected utility based on information from the agent's past experiences and is not about being all knowing. The new sensor helps the agent get information concerning traps when it comes across them, and thus allows the design of a more developed agent that is able to bypass such cells with ease.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic Vacuum World Exploit was giving the agent +1 point every time it activated its suction motor and collected a unit of dirt. A rational agent exploited this internal metric by cleaning a patch of dirt, dumping it back onto the floor, and cleaning the same spot over and over to harvest infinite points, which showed why performance measures must be tied to real external outcomes instead of internal actions.

Finalize and Lock Lab 01 Analytical Evaluation